

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
— o0o —



BÁO CÁO BÀI TẬP LỚN

Học phần: Nguyên lý hệ điều hành

(Mã học phần: IT 3070)

MyShell

Giảng viên hướng dẫn: TS. Phạm Đăng Hải

Sinh viên thực hiện : Nguyễn Đào Nam Hải - 20235321

Lớp : IT3070 - 157503

Hà Nội - 2025

Mục lục

1	Giới thiệu đề tài	3
2	Nội dung chính của đề tài	3
2.1	Tổng quan về MyShell	3
2.1.1	Tiến trình	3
2.1.2	Mục tiêu và chức năng	3
2.1.3	Cấu trúc chương trình	4
2.2	Thực thi và quản lý tiến trình	6
2.2.1	Tạo tiến trình	6
2.2.2	Quản lý tiến trình nền	7
2.3	Khó khăn và giải pháp	7
2.3.1	Khó khăn	7
2.3.2	Giải pháp	8
3	Ví dụ chạy thử MyShell	8
4	Kết luận	12

1 Giới thiệu đề tài

Shell là công cụ quan trọng trong khoa học máy tính, kết nối người dùng với hệ điều hành để thực thi lệnh và quản lý tiến trình. Trên Windows, Command Prompt là một ví dụ điển hình, nhưng việc tạo shell tùy chỉnh giúp hiểu sâu hơn về hệ điều hành và đáp ứng nhu cầu riêng.

Dự án "MyShell" phát triển một shell đơn giản, hiệu quả trên Windows bằng ngôn ngữ C và API `CreateProcessA`. MyShell hỗ trợ chạy chương trình, quản lý tiến trình nền với các lệnh như `list`, `stop`, `resume`, `kill`, và xử lý file batch (.bat) để tăng tính linh hoạt.

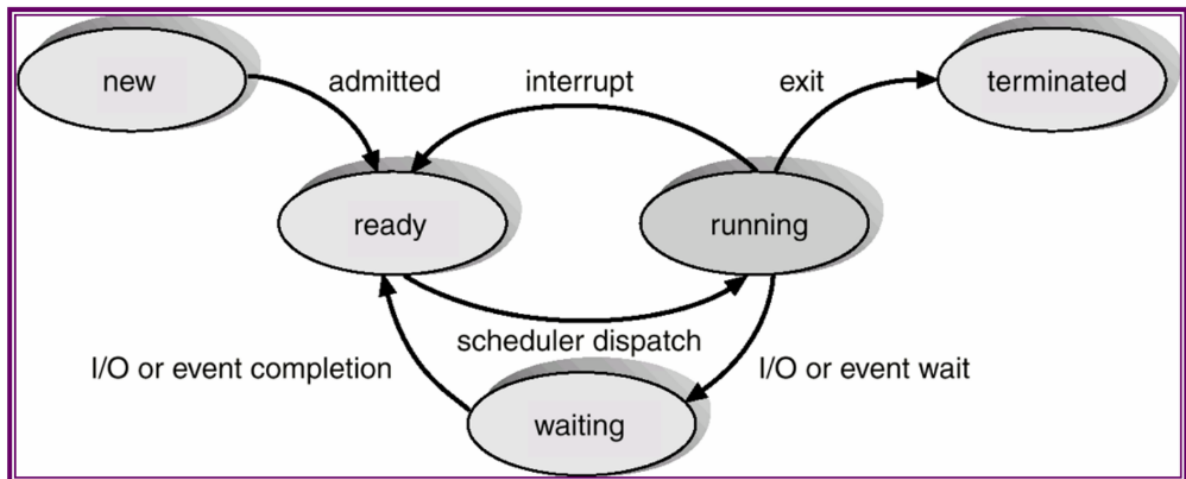
Báo cáo này trình bày quá trình thiết kế, triển khai MyShell, các chức năng chính và thách thức khi tích hợp với Windows. Dự án nhằm tạo công cụ thực thi lệnh, đồng thời là nền tảng học tập về quản lý tiến trình, mang lại giá trị thực tiễn và nâng cao hiểu biết về hệ điều hành.

2 Nội dung chính của đề tài

2.1 Tổng quan về MyShell

2.1.1 Tiến trình

Tiến trình là các chương trình đang được thực thi. Mỗi tiến trình được thể hiện trong hệ thống bởi khối điều khiển quy trình (PCBs - Process Control Blocks) - một cấu trúc dữ liệu trong nhân hệ điều hành chứa thông tin về một hệ thống tiến trình cụ thể. Tiến trình sở hữu một bộ đếm chương trình (program counter), tức là thanh ghi chứa địa chỉ của lệnh tiếp theo được CPU thực thi.



2.1.2 Mục tiêu và chức năng

MyShell là một shell tùy chỉnh trên Windows, cho phép người dùng thực thi lệnh, quản lý tiến trình và chạy file batch. Các chức năng chính bao gồm:

- Thực thi chương trình cơ bản.
- Quản lý tiến trình nền (background processes).
- Hỗ trợ lệnh: `list` (liệt kê tiến trình), `kill` (kết thúc), `stop` (dừng), `resume` (tiếp tục).
- Xử lý file batch (.bat).

2.1.3 Cấu trúc chương trình

MyShell được viết bằng C, sử dụng API Windows như `CreateProcessA` để tạo tiến trình. Chương trình gồm các thành phần chính:

- **Phân tích lệnh:** Hàm `parse_command` tách chuỗi lệnh thành mảng tham số.

```
//Phân tích lệnh
int parse_command(char *input, char **args) {
    int argc = 0;
    char *token = strtok(input, " \n");
    while (token != NULL && argc < MAX_ARGS) {
        args[argc++] = token;
        token = strtok(NULL, " \n");
    }
    args[argc] = NULL; //Phần tử cuối là NULL vì CreateProcessA yêu cầu kết thúc là NULL
    return argc;
}
```

Hình 1: Phân tích lệnh

Khi nhập vào 1 chuỗi lệnh thì MyShell sẽ cần phân tích chuỗi lệnh đó thành những phần riêng lẻ để máy tính có thể phân biệt được đâu là tên chương trình, đâu là tham số. Hệ điều hành cần biết đâu là chương trình và phải truyền tham số cho chương trình.

=> Hàm `parse_command()` có chức năng nhận đầu vào là một chuỗi lệnh, sau đó sử dụng hàm `strtok()` để tách chuỗi thành từng phần riêng biệt dựa trên ký tự phân cách là `\n`. Các phần này sẽ được lưu vào mảng `args` để truyền cho hàm `CreateProcessA`. Đồng thời, hàm đảm bảo rằng danh sách tham số kết thúc bằng giá trị `NULL` và cuối cùng trả về số lượng tham số đã được tách ra từ chuỗi lệnh ban đầu.

- **Kiểm tra background:** Hàm `is_background` xác định tiến trình chạy nền bằng ký tự `"&"`.

```
36 // Hàm kiểm tra xem có chạy background không (dấu &)
37 int is_background(char **args, int argc) {
38     if (argc > 0 && strcmp(args[argc - 1], "&") == 0) {
39         args[argc - 1] = NULL; // Xóa dấu & khỏi danh sách tham số
40         return 1;
41     }
42     return 0;
43 }
```

Hình 2: Kiểm tra có sử dụng background không

Khi người dùng nhập một lệnh, MyShell cần xác định xem lệnh đó có yêu cầu chạy ở chế độ nền hay không. Điều này giúp hệ điều hành biết liệu có cần chờ tiến trình kết thúc hay không.

=> Hàm `is_background()` kiểm tra xem tham số cuối của lệnh có phải là ký tự `&` hay không. Nếu có, hàm sẽ loại bỏ ký tự này khỏi danh sách tham số bằng cách gán `NULL` cho phần tử cuối, đồng thời trả về 1 để báo hiệu rằng tiến trình cần chạy nền. Nếu không có ký tự `&`, hàm trả về 0, nghĩa là tiến trình sẽ chạy ở chế độ thông thường.

- **Thực thi lệnh:** Hàm `execute_command` xử lý lệnh và quản lý tiến trình.

```

45 //Thực hiện lệnh mà người dùng yêu cầu
46 void execute_command(char **args, int background) {
47     //Xử lý các lệnh quản lý shell
48     if (strcmp(args[0], "exit") == 0) {
49         printf("Exiting myShell...\n");
50         exit(0);
51     } else if (strcmp(args[0], "list") == 0) {
52         printf("List of background processes:\n");
53         for (int i = 0; i < process_count; i++) {
54             printf("PID: %lu, Name: %s, Status: %s\n", process_list[i].pid, process_list[i].name, process_list[i].status);
55         }
56         return;
57     } else if (strcmp(args[0], "kill") == 0) {
58         if (args[1] == NULL) {
59             printf("Usage: kill <pid>\n");
60             return;
61         }
62         DWORD pid = atoi(args[1]);
63         for (int i = 0; i < process_count; i++) {
64             if (process_list[i].pid == pid) {
65                 TerminateProcess(process_list[i].hProcess, 1);
66                 strcpy(process_list[i].status, "Terminated");
67                 printf("Process %lu terminated\n", pid);
68                 return;
69             }
70         }
71         printf("Process %lu not found\n", pid);
72         return;
73     } else if (strcmp(args[0], "stop") == 0) {
74         if (args[1] == NULL) {
75             printf("Usage: stop <pid>\n");
76             return;
77         }
78         DWORD pid = atoi(args[1]);
79         for (int i = 0; i < process_count; i++) {
80             if (process_list[i].pid == pid) {
81                 SuspendThread(process_list[i].hProcess);
82                 strcpy(process_list[i].status, "Stopped");
83                 printf("Process %lu stopped\n", pid);
84                 return;
85             }
86         }
87         printf("Process %lu not found\n", pid);
88         return;
89     } else if (strcmp(args[0], "resume") == 0) {
90         if (args[1] == NULL) {
91             printf("Usage: resume <pid>\n");
92             return;
93         }
94         DWORD pid = atoi(args[1]);
95         for (int i = 0; i < process_count; i++) {
96             if (process_list[i].pid == pid) {
97                 ResumeThread(process_list[i].hProcess);
98                 strcpy(process_list[i].status, "Running");
99                 printf("Process %lu resumed\n", pid);
100                 return;
101             }
102         }
103         printf("Process %lu not found\n", pid);
104         return;
105     }

```

Hình 3: Thực hiện lệnh mà người dùng yêu cầu

Khi người dùng nhập một lệnh, MyShell cần khởi chạy tiến trình tương ứng để thực thi lệnh đó. Nếu lệnh hợp lệ, hệ điều hành sẽ tạo một tiến trình mới để thực thi nó.

=> Hàm `execute_command()` có nhiệm vụ gọi `CreateProcessA()` để khởi chạy tiến trình mới

với các tham số đã được phân tích trước đó. Nếu lệnh yêu cầu chạy nền (xác định bởi hàm `is_background()`), tiến trình sẽ được khởi chạy mà không cần đợi hoàn thành. Nếu không, chương trình cha sẽ chờ tiến trình con kết thúc trước khi tiếp tục thực thi lệnh tiếp theo.

- **Danh sách tiến trình:** Mảng `process_list` lưu thông tin tiến trình (PID, tên, trạng thái).

```

9 //Thông tin tiến trình con
10 typedef struct {
11     HANDLE hProcess; //tham chiếu đến tiến trình con
12     DWORD pid;       //số hiệu của tiến trình
13     char name[MAX_INPUT]; //tên lệnh or chương trình thực thi
14     char status[20];    //Lưu trạng thái tiến trình
15 } Process;
16
17 Process process_list[100]; // Danh sách tiến trình con
18 int process_count = 0;     //Đếm số chương trình con trong list

```

Hình 4: Thông tin về tiến trình

Khi một tiến trình mới được tạo, thông tin của nó bao gồm Process ID (PID), tên lệnh và trạng thái sẽ được lưu vào mảng `process_list`. Hệ thống sẽ sử dụng danh sách này để theo dõi và quản lý các tiến trình đang chạy hoặc đã kết thúc. => Cấu trúc `Process` chứa các trường cần thiết như `HANDLE hprocess` (tham chiếu tiến trình con), `DWORD pid` (số hiệu tiến trình), `char name[MAX_INPUT]` (tên lệnh) và `char status[20]` (trạng thái tiến trình). Biến `process_count` dùng để đếm số tiến trình trong danh sách.

2.2 Thực thi và quản lý tiến trình

2.2.1 Tạo tiến trình

MyShell sử dụng `CreateProcessA` để tạo tiến trình con từ lệnh người dùng nhập. Quy trình bao gồm:

- Chuẩn bị chuỗi lệnh từ mảng tham số.
- Kiểm tra file `.bat` và thêm `"cmd.exe /c"` nếu cần.
- Gọi `CreateProcessA` để thực thi.

Tiến trình có thể chạy foreground (chờ hoàn thành) hoặc background (chạy ngầm).

```

125 //Tạo tiến trình con
126 BOOL success = CreateProcessA(NULL, command_line, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi);
127 if (!success) {
128     printf("Failed to execute command: %lu\n", GetLastError()); //In ra màn hình báo lỗi nếu tạo tiến trình thất bại
129     return;
130 }

```

Hình 5: Tạo tiến trình bằng `CreateProcessA` trong MyShell

2.2.2 Quản lý tiến trình nền

MyShell hỗ trợ quản lý tiến trình nền với các lệnh:

- **list**: Hiển thị PID, tên và trạng thái của các tiến trình.
- **kill <pid>**: Kết thúc tiến trình bằng `TerminateProcess`.
- **stop <pid>**: Tạm dừng tiến trình bằng `SuspendThread`.
- **resume <pid>**: Tiếp tục tiến trình bằng `ResumeThread`.

Thông tin tiến trình được lưu trong mảng `process_list` với tối đa 100 phần tử.

```
WELCOME TO MY SHELL
The following commands:
list: List all background processes (ID, name, status)
kill <pid>: Terminate a process
stop <pid>: Stop a process
resume <pid>: Resume a process
exit: Exit my shell program
myShell> calc &
Child process 13120 running in background
myShell> notepad &
Child process 19964 running in background
myShell> chrome &
Failed to execute command: 2
myShell> b5_C.exe
Failed to execute command: 2
myShell> list
List of background processes:
PID: 13120, Name: calc, Status: Running
PID: 19964, Name: notepad, Status: Running
myShell>
```

Hình 6: Danh sách tiến trình nền trong MyShell

2.3 Khó khăn và giải pháp

2.3.1 Khó khăn

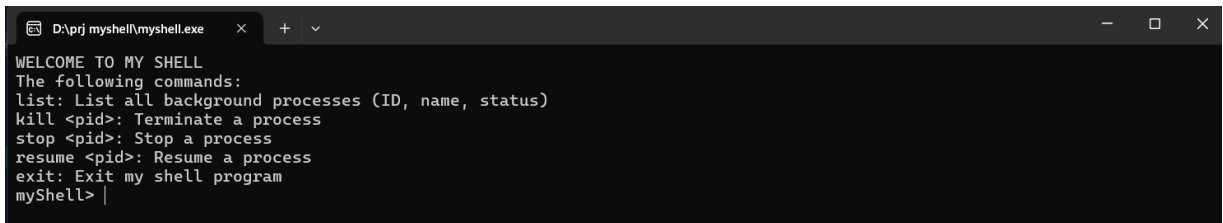
- **Tích hợp API Windows**: Sử dụng `CreateProcessA` đòi hỏi xử lý chuỗi ANSI và cấu trúc `PROCESS_INFORMATION` chính xác.
- **Quản lý trạng thái**: Đồng bộ trạng thái tiến trình (Running, Stopped, Terminated) trong danh sách.
- **Giới hạn tài nguyên**: Chỉ hỗ trợ tối đa 100 tiến trình nền.

2.3.2 Giải pháp

- Dùng `CreateProcessA` với chuỗi ANSI để đơn giản hóa, thay vì chuyển sang Unicode.
- Cập nhật trạng thái thủ công sau mỗi lệnh `kill`, `stop`, `resume`.
- Kiểm tra `process_count` trước khi thêm tiến trình mới.

3 Ví dụ chạy thử MyShell

- Khi bắt đầu sử dụng MyShell sẽ hiện ra giao diện:

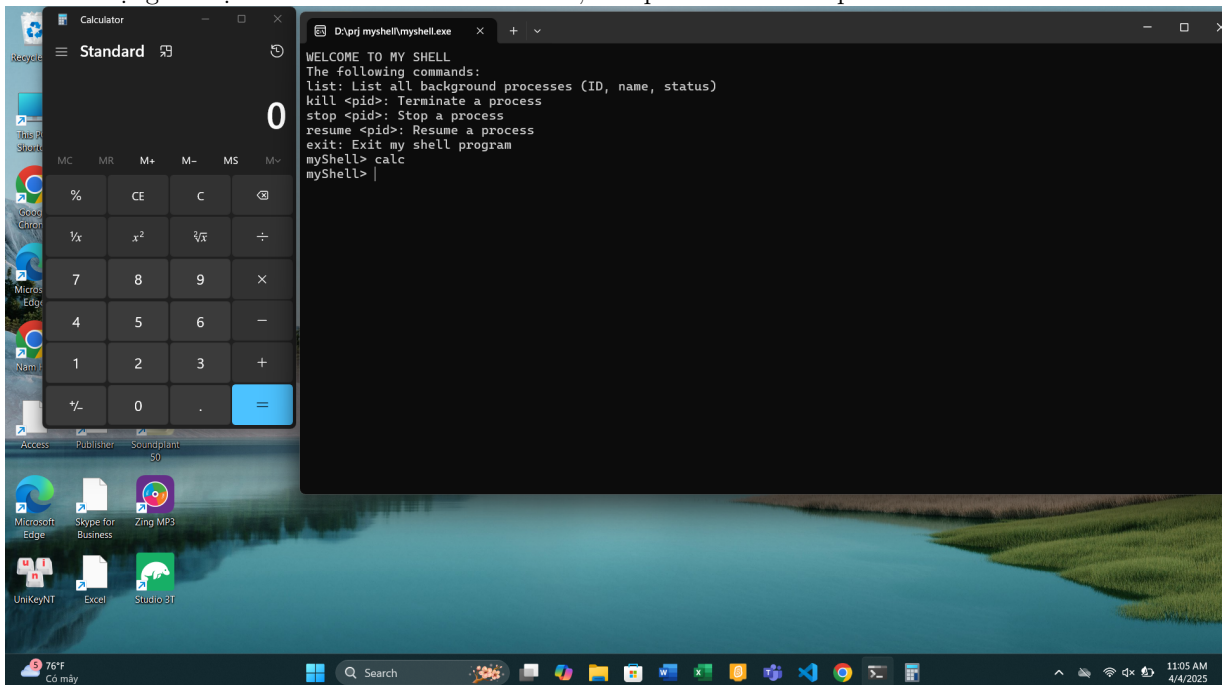


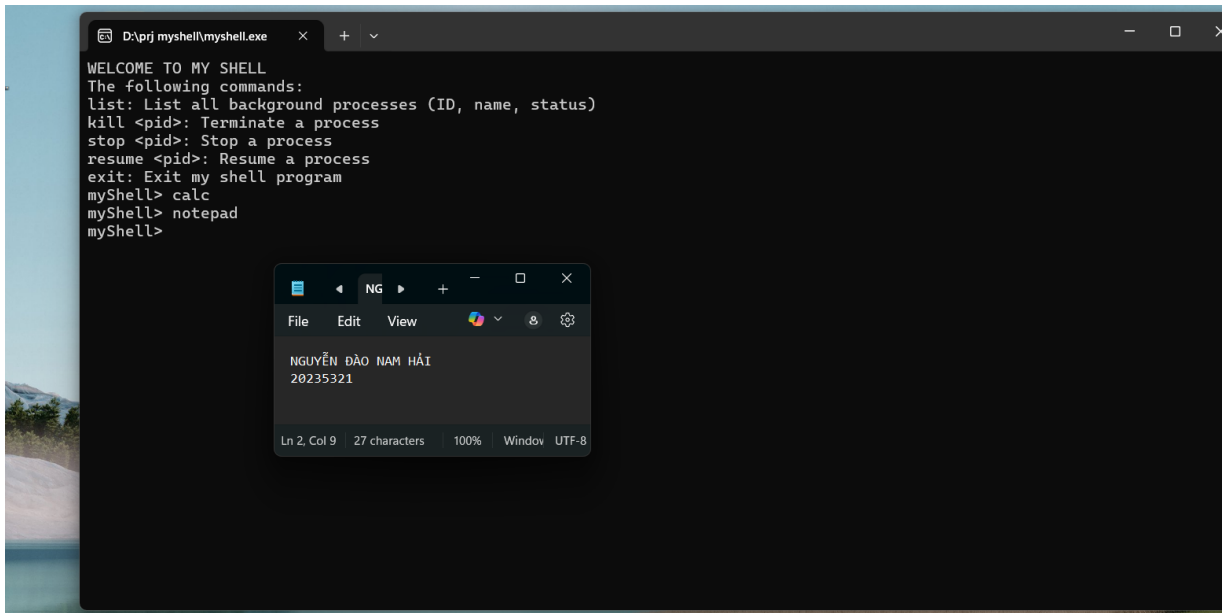
```

D:\prj myshell\myshell.exe
WELCOME TO MY SHELL
The following commands:
list: List all background processes (ID, name, status)
kill <pid>: Terminate a process
stop <pid>: Stop a process
resume <pid>: Resume a process
exit: Exit my shell program
myShell> |
    
```

Hình 7: Giao diện MyShell

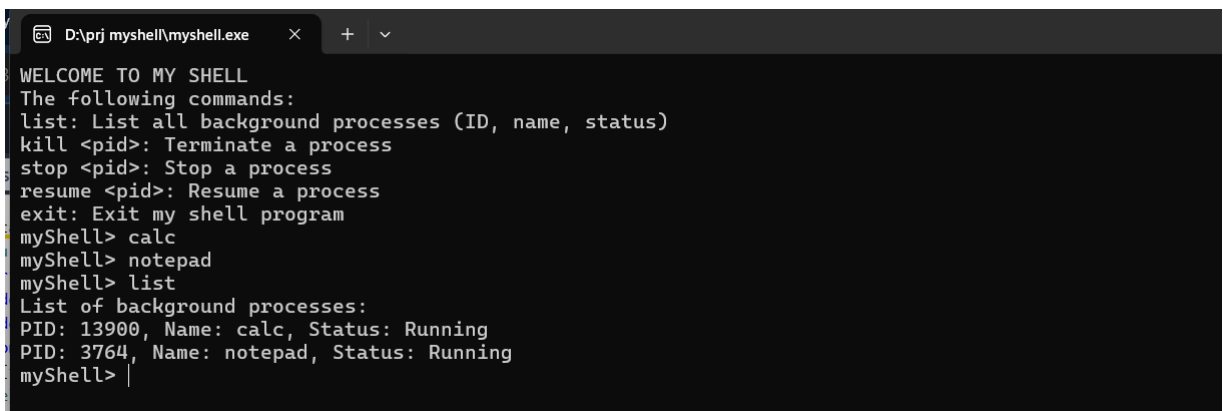
- Sử dụng các lệnh như: `calc` để mở calculator, `notepad` để mở notepad





Hình 8: Khi truy cập calculator và notepad thông qua MyShell

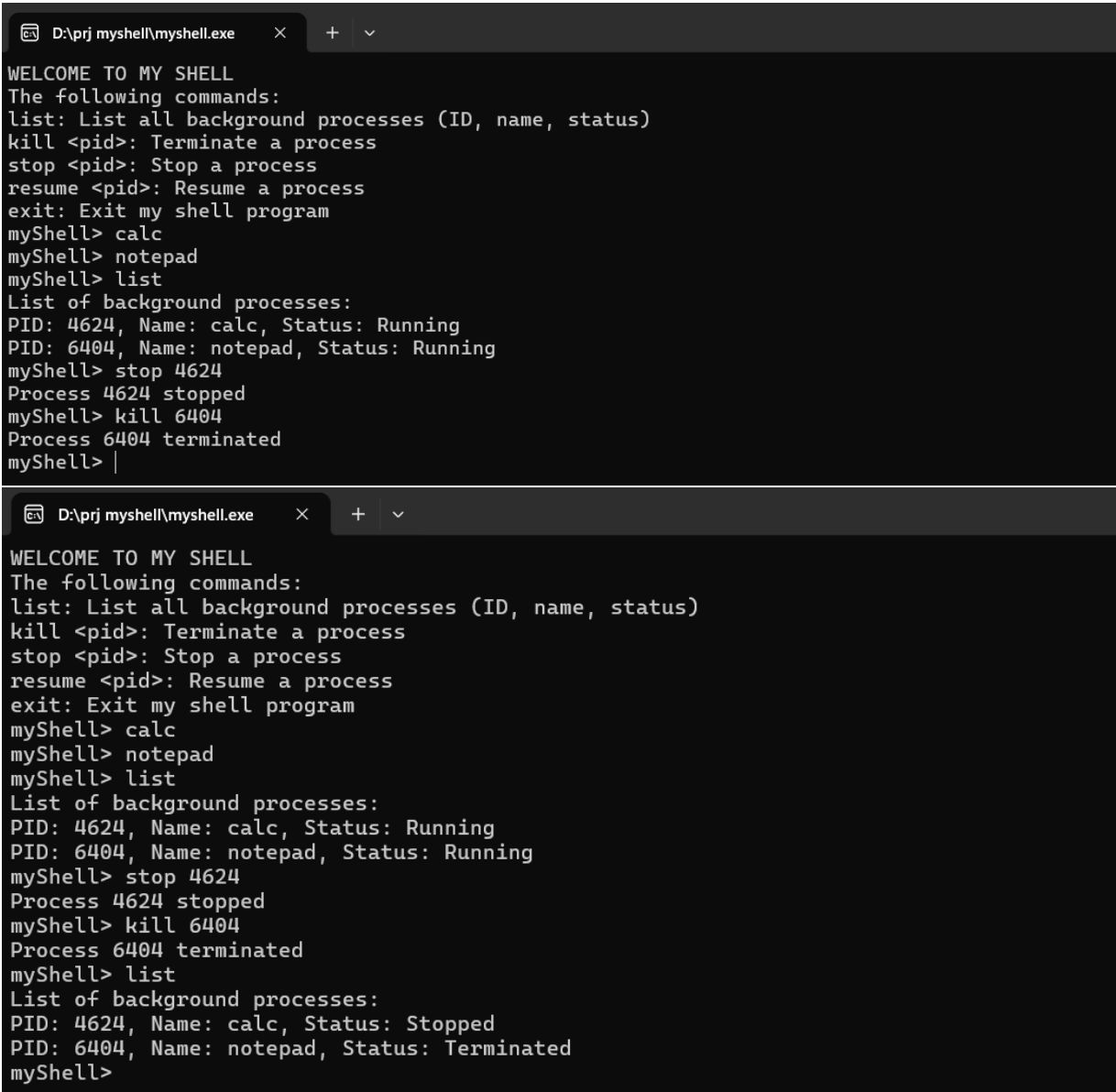
- Sử dụng câu lệnh list để xem các tiến trình đang chạy:



Hình 9: các tiến trình đang chạy

Ta thấy rằng có 2 tiến trình đang ở trạng thái Running đó là tiến trình 4624 có Name:calc và tiến trình 6404 có Name: notepad

- Sử dụng câu lệnh stop và kill



The image contains two screenshots of a terminal window titled "D:\prj myshell\myshell.exe". The window shows the execution of a custom shell program. The first screenshot shows the initial state where the shell lists running processes (calc and notepad) and then stops the calc process (PID 4624) and kills the notepad process (PID 6404). The second screenshot shows the state after these actions, where the calc process is now "Stopped" and the notepad process is "Terminated".

```

WELCOME TO MY SHELL
The following commands:
list: List all background processes (ID, name, status)
kill <pid>: Terminate a process
stop <pid>: Stop a process
resume <pid>: Resume a process
exit: Exit my shell program
myShell> calc
myShell> notepad
myShell> list
List of background processes:
PID: 4624, Name: calc, Status: Running
PID: 6404, Name: notepad, Status: Running
myShell> stop 4624
Process 4624 stopped
myShell> kill 6404
Process 6404 terminated
myShell> |

WELCOME TO MY SHELL
The following commands:
list: List all background processes (ID, name, status)
kill <pid>: Terminate a process
stop <pid>: Stop a process
resume <pid>: Resume a process
exit: Exit my shell program
myShell> calc
myShell> notepad
myShell> list
List of background processes:
PID: 4624, Name: calc, Status: Running
PID: 6404, Name: notepad, Status: Running
myShell> stop 4624
Process 4624 stopped
myShell> kill 6404
Process 6404 terminated
myShell> list
List of background processes:
PID: 4624, Name: calc, Status: Stopped
PID: 6404, Name: notepad, Status: Terminated
myShell>
  
```

Hình 10: Sau khi sử dụng stop tiến trình 4624 và kill tiến trình 6404

Tiến trình 4624 đã dừng lại và tiến trình 6424 đã hoàn thành

- Sử dụng câu lệnh resume và exit

```

WELCOME TO MY SHELL
The following commands:
list: List all background processes (ID, name, status)
kill <pid>: Terminate a process
stop <pid>: Stop a process
resume <pid>: Resume a process
exit: Exit my shell program
myShell> calc
myShell> notepad
myShell> list
List of background processes:
PID: 4624, Name: calc, Status: Running
PID: 6404, Name: notepad, Status: Running
myShell> stop 4624
Process 4624 stopped
myShell> kill 6404
Process 6404 terminated
myShell> list
List of background processes:
PID: 4624, Name: calc, Status: Stopped
PID: 6404, Name: notepad, Status: Terminated
myShell> resume 4624
Process 4624 resumed
myShell> list
List of background processes:
PID: 4624, Name: calc, Status: Running
PID: 6404, Name: notepad, Status: Terminated
myShell>

```

Hình 11: resume 4624

Tiến trình 4624 từ trạng thái stopped chuyển thành running. Khi nhập exit sẽ thoát khỏi MyShell

4 Kết luận

Dự án MyShell đã thành công trong việc xây dựng một shell mạnh mẽ trên hệ điều hành Windows, mang đến khả năng thực thi lệnh và quản lý tiến trình một cách hiệu quả. Dự án không chỉ đáp ứng đầy đủ các yêu cầu học tập mà còn mở ra nhiều cơ hội để phát triển thêm các tính năng mới, như quản lý ưu tiên tiến trình hoặc tích hợp giao diện đồ họa, giúp người dùng có thể tương tác trực quan hơn với hệ thống.

Thông qua quá trình phát triển và triển khai, tôi đã có cơ hội làm quen và áp dụng các kỹ thuật lập trình hệ thống, đặc biệt là việc làm việc với các API của Windows để quản lý tiến trình và tài nguyên hệ thống. Các lệnh như tạo tiến trình, dừng và tiếp tục tiến trình, cùng với việc quản lý tiến trình nền, không chỉ giúp tôi nắm vững cách thức hoạt động của hệ điều hành mà còn rèn luyện kỹ năng giải quyết vấn đề và tối ưu hóa mã nguồn.

Ngoài ra, dự án này còn giúp tôi hiểu sâu hơn về các vấn đề liên quan đến đồng bộ hóa tiến trình và các tác vụ hệ thống phức tạp, điều này sẽ là nền tảng vững chắc cho những nghiên cứu và phát triển phần mềm trong tương lai. Đây là một bước đi quan trọng trong việc nâng cao kỹ năng lập trình hệ thống và phát triển ứng dụng thực tế, từ đó mở ra những cơ hội học hỏi và ứng dụng rộng rãi trong các dự án tương lai.

Tài liệu

- [1] **Hart, J. M.** (2011). *Windows System Programming*, Addison-Wesley.
- [2] **Microsoft Docs.** (2023). *CreateProcessA function*, <https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa>.